1.two sum

```
class Solution:
    def twoSum(self, nums, target):
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] + nums[j] == target:
                    return [i, j]
```

Input:

nums =[2,7,11,15]

target =9

Output

[0,1]

Expected

[0,1]

2.add two numbers

```
# Definition for singly-linked list.
# class ListNode(object):
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution(object):
    def addTwoNumbers(self, l1, l2):
        sum = ListNode()
        tail = sum
        carry = 0

        while l1 or l2 or carry:
            val1 = l1.val if l1 else 0
            val2 = l2.val if l2 else 0
```

```
            total = val1 + val2 + carry
            carry = total//10

            tail.next = ListNode(total%10)
            tail = tail.next

            if l1: l1 = l1.next
            if l2: l2 = l2.next

        return sum.next
```

Input:

l1 =[2,4,3]

l2 =[5,6,4]

Output:

[7,0,8]

Expected

[7,0,8]

3.Longest substring without repeating characters

```
class Solution(object):
    def lengthOfLongestSubstring(self, s):
        ans = 0
        i = 0
        j = 0

        while j != len(s):
            if len(set(s[i:j+1])) != len(s[i:j+1]):
                i += 1
            else:
```

```
            if len(s[i:j+1]) > ans:
                ans = len(s[i:j+1])
            j += 1
        return ans
```

Input:

s ="abcabcbb"

Output:

3

Expected

3

4.median of two sorted arrays

```
class Solution:
    def findMedianSortedArrays(self, nums1, nums2):
        n, m = len(nums1), len(nums2)
        total = n + m

        # Indices of median positions
        mid1 = (total - 1) // 2
        mid2 = total // 2

        i = j = count = 0
        left = right = -1

        # Merge arrays until reaching median indices
        while count <= mid2:
            # Pick smaller element from nums1 or nums2
            if i < n and (j >= m or nums1[i] <= nums2[j]):
                val = nums1[i]
                i += 1
            else:
```

```
            val = nums2[j]
            j += 1

        # Store elements at median positions
        if count == mid1:
            left = val
        if count == mid2:
            right = val

        count += 1

    # Odd -> right, Even -> average of left and right
    if total % 2 == 1:
        return right
    return (left + right) / 2.0
```

Input:

nums1 =[1,3]

nums2 =[2]

Output:

2.00000

Expected

2.00000

5.longest palindrome substring

```
class Solution(object):
    def longestPalindrome(self, s):
        res = ""

        for i in range(len(s)):
            l, r = i, i
            while l >= 0 and r < len(s) and s[l] == s[r]:
```

```
            if len(s[l:r+1]) > len(res):
                res = s[l:r+1]
            l -= 1
            r += 1

        l, r = i, i + 1
        while l >= 0 and r < len(s) and s[l] == s[r]:
            if len(s[l:r+1]) > len(res):
                res = s[l:r+1]
            l -= 1
            r += 1

    return res
```

Input:

s ="babad"

Output:

"bab"

Expected

"bab"

6.zigzag conversion

```
class Solution(object):
    def convert(self, s, numRows):
        if numRows==1:
            return s
        T=['']*numRows
        R,i=-1,0
        while i!=len(s):
            while R!=numRows-1 and i!=len(s):
                R+=1
                T[R]+=s[i]
```

```
            i+=1
        while R!=0 and i!=len(s):
            R-=1
            T[R]+=s[i]
            i+=1
    ch=''
    for i in range(len(T)):
        ch+=T[i]
    return ch
```

Input:

s ="PAYPALISHIRING"

numRows =3

Output:

"PAHNAPLSIIGYIR"

Expected

"PAHNAPLSIIGYIR"

7.reverse integer

```
class Solution:
    # @return an integer
    def reverse(self, x):
        result = 0

        if x < 0:
            symbol = -1
            x = -x
        else:
            symbol = 1

        while x:
            result = result * 10 + x % 10
```

```
        x /= 10

    return 0 if result > pow(2, 31) else result * symbol
```

Input:

x =123

Output:

321

Expected

321

8.string to integer

```
class Solution(object):
    def myAtoi(self, s):
        """
        :type s: str
        :rtype: int
        """
        i = 0
        n = len(s)

        while i < n and s[i] == ' ':
            i += 1

        sign = 1
        if i < n and (s[i] == '-' or s[i] == '+'):
            sign = -1 if s[i] == '-' else 1
            i += 1

        result = 0
        while i < n and s[i].isdigit():
```

```
            result = result * 10 + int(s[i])
            if result * sign > 2**31 - 1:
                return 2**31 - 1
            if result * sign < -2**31:
                return -2**31
            i += 1

        return result * sign
```

Input:

s ="42"

Output:

42

Expected

42

9.regular expression matching

```
class Solution(object):
    def is Match(self, s, p):
        """
        :type s: str
        :type p: str
        :rtype: bool
        """
        m, n = len(s), len(p)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        for j in range(1, n + 1):
            if p[j - 1] == '*':
                dp[0][j] = dp[0][j - 2]
```

```
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if p[j - 1] in {s[i - 1], '.'}:
                    dp[i][j] = dp[i - 1][j - 1]
                elif p[j - 1] == '*':
                    dp[i][j] = dp[i][j - 2]
                    if p[j - 2] in {s[i - 1], '.'}:
                        dp[i][j] = dp[i][j] or dp[i - 1][j]


        return dp[m][n]
```

Input:

s ="aa"

p ="a"

Output:

false

Expected

false

10. roman to integer

```
class Solution:
    def romanToInt(self, s):
        roman = {'I': 1, 'V': 5, 'X': 10, 'L': 50,
                 'C': 100, 'D': 500, 'M': 1000}
        total = 0
        prev = 0

        for char in reversed(s):
            value = roman[char]
            if value < prev:
                total -= value
            else:
```

```
            total += value
        prev = value


    return total
```

Input:

s ="III"

Output:

3

Expected

3

11.Remove element

```
    class Solution(object):
  def removeElement(self, nums, val):
    ptr =0
    for i in range(0,len(nums)):
        if nums[i]!=val:
            nums[ptr] =nums[i]
            ptr = ptr +1
    return  ptr
```

input:

nums = [0, 1, 2, 2, 3, 0, 4, 2]

val = 2

output:

New length: 5

Modified array: [0, 1, 3, 0, 4]

12. 3 sum

```
class Solution:
    def threeSum(self, nums):
        nums.sort()
        T = []
        for i in range(len(nums) - 2):
            if 0 < i and nums[i] == nums[i - 1]:
                continue
            l, r = i + 1, len(nums) - 1
            while l < r:
                if nums[i] + nums[l] + nums[r] > 0:
                    r -= 1
                elif nums[i] + nums[l] + nums[r] < 0:
                    l += 1
                else:
                    T.append([nums[i], nums[l], nums[r]])
                    l += 1
                    r -= 1
                    while l < r and nums[l] == nums[l - 1]:
                        l += 1
                    while l < r and nums[r] == nums[r + 1]:
                        r -= 1
```

input:

```
  nums = [-1, 0, 1, 2, -1, -4]
sol = Solution()
print(sol.threeSum(nums))
```

output:

```
[[-1, -1, 2], [-1, 0, 1]]
```